

VIT BHOPAL UNIVERSITY

PROJECT REPORT

Smart Complaint Management System

Using Java Collections and Object-Oriented Programming

Submitted by

AVANI KHARE

Computer Science and Engineering

Artificial Intelligence and Machine Learning

Submitted through the VITyarthi Platform

Academic Year 2026

Acknowledgement

Working on this project turned out to be a lot more involved than I expected when I first picked the topic, and I would not have finished it without a fair bit of help along the way.

I am grateful to my faculty members and course instructors, who were patient with my questions and had a way of explaining things that made the concepts finally click. A good number of the design decisions in this project came out of those conversations.

I would also like to thank VIT Bhopal University for the lab facilities and the learning environment that made it possible to build and test the application properly.

Finally, thanks to my classmates and friends, who tested the program and pointed out the inputs that broke it, and to my family, who put up with me while I was stuck on bugs.

Abstract

Almost every hostel, department office and service desk deals with complaints, and most of them still handle those complaints on paper. Registers get lost, the same issue gets written down twice, and nobody can say with certainty whether a complaint from last month was ever resolved. This project is an attempt to fix that problem at a small scale.

The Smart Complaint Management System is a console-based Java application that lets people register, store, search and track complaints. It has two sides to it. A normal user can create an account, log in, file a complaint, look at the complaints they have filed, search for a specific one by its ID, and check the most recent complaint in the system. An administrator logs in separately and gets a dashboard for reviewing complaints, changing their status and priority, and keeping an eye on what is still pending.

Underneath, the project is really an exercise in putting core Java to work. Classes and objects model the users, admins and complaints; encapsulation and access modifiers keep the data from being modified carelessly; the Collections Framework holds the records in memory; enums pin down fixed values like priority and status so that an invalid value cannot sneak in; exception handling keeps a bad input from crashing the whole program; and file handling writes everything to disk so the records survive after the program closes.

The result is a small system, but a complete one, and it shows how a handful of Java concepts fit together when they are used to solve an actual problem instead of being practised in isolation.

Table of Contents

1	Introduction
2	Problem Statement
3	Objectives
4	The Existing Approach
5	The Proposed System
6	Technologies and Requirements
7	Java Concepts Used
8	Features and Modules
9	System Design and Working
10	Project Structure
11	Data Storage
12	Testing
13	Advantages
14	Limitations
15	Future Scope
16	Conclusion
17	References

1. Introduction

Complaint handling is one of those routine tasks that looks trivial until you have to do it at scale. In a hostel with a few hundred students, the water cooler on the second floor, a flickering tube light and a dead WiFi access point can all be reported in the same afternoon, sometimes by several people at once. When all of this goes into a register at the warden's desk, a few predictable things happen: entries get repeated, older complaints drift out of sight, and there is no reliable way to answer the simple question of what is still pending.

This project takes that situation and rebuilds it as software. The Smart Complaint Management System is a console application written in Java. Users create an account, log in and file complaints; every complaint gets a unique ID the moment it is created, so it can be tracked afterwards. An administrator logs in through a separate route and can see everything, update the status of a complaint, change its priority and check which complaints are still open.

The choice to keep it on the console was deliberate. The point of the project was to get the logic, the class design and the data handling right, rather than to spend the available time on an interface. Everything visible in the program is built out of core Java, without any external framework.

2. Problem Statement

A manual complaint system usually means a register, a shared spreadsheet, or a stack of loose slips of paper. Any of these works well enough on a slow day and falls apart on a busy one. The specific problems that motivated this project were:

- Records are hard to maintain. Pages fill up, handwriting varies, and older registers get put away and forgotten.
- Searching is slow. Finding a single complaint means flipping through everything until it turns up.
- Complaint numbers are unreliable. The same number can be written twice, or a number can be skipped entirely, and nobody notices until much later.
- Status and priority are invisible. There is usually no clear way to tell an urgent complaint from a minor one, or a resolved complaint from an open one.
- Monitoring takes time. Working out what is still pending means reading the whole register again.
- User records are separate from complaint records, so there is no easy way to see everything one person has reported.

What is needed is a straightforward computerised system that registers a complaint, gives it an identity, stores it safely and lets anyone look it up again in a couple of seconds.

3. Objectives

The goals set at the start of the project were the following:

- Build a complaint management system that is simple enough for a first-time user to operate without instructions.
- Let users register complaints in a few steps.
- Generate a unique complaint ID automatically, so no two complaints can collide.
- Provide search and tracking, so a complaint can be located by its ID.
- Manage priority and status properly, using fixed values instead of free text.
- Apply object-oriented programming to a real problem rather than to a textbook exercise.
- Use the Java Collections Framework to hold and manipulate the records in memory.
- Persist users and complaints to files so nothing is lost when the program exits.
- Keep user and administrator functionality clearly separated.

4. The Existing Approach

What is in place at the moment, in most small setups, is not really a system at all. Complaints are written in registers, typed into a spreadsheet that one person maintains, or kept as loose text records. To file a complaint, a person often has to find the administrator in person; to check on it later, they have to find the administrator again.

The weaknesses follow from that. Nothing is automated, so every step depends on someone remembering to do it. Searching is manual and therefore slow. Records are not organised in any consistent way, since the format depends on whoever wrote the entry. And because status is rarely recorded, the current state of a complaint is often known only to the person handling it.

5. The Proposed System

The proposed system replaces the register with a Java console application. The logic is organised into model classes, service classes and menu classes, so that the code that stores and processes complaints is kept separate from the code that displays menus. Users and administrators go through different entry points and see different options.

The features it provides are:

- User registration and login with a username, password and name.
- A separate admin login leading to its own dashboard.
- Automatic generation of a unique complaint ID for every new complaint.
- Complaints that carry a title, description, category, the username of the person who filed them, a priority and a status.

- The ability to view only your own complaints, or all complaints in the system.
- Search by complaint ID.
- A shortcut to view the most recently registered complaint.
- File-based storage, so both users and complaints are still there the next time the program is started.

6. Technologies and Requirements

The project uses a deliberately small toolchain. Everything listed below is either part of a standard JDK installation or free to set up.

Component	Details
Programming Language	Java
Programming Paradigm	Object-Oriented Programming
Data Structures	Java Collections Framework
Storage	File handling using plain text files
IDE	Visual Studio Code
Version Control	Git
Repository Hosting	GitHub
Operating System	Windows
Execution Type	Console application

6.1 Hardware Requirements

Nothing unusual is needed here. Any computer or laptop that can run a JDK comfortably will do. In practice that means around 4 GB of RAM or more, a keyboard and monitor, and enough free storage for the JDK, the project files and the small data files the program writes.

6.2 Software Requirements

A Java Development Kit must be installed and available on the system path. Visual Studio Code was used for development, though any Java IDE or even a plain text editor with the command line would work equally well. PowerShell or the command prompt is needed to compile and run the program, and Git along with a GitHub account was used to keep the code under version control.

7. Java Concepts Used

7.1 Object-Oriented Programming

The application is built around classes that correspond to the things it actually deals with: a User, an Admin and a Complaint. Each of these keeps its data private and exposes it through methods, which means a complaint's ID cannot be changed accidentally from somewhere else in the program. Constructors set up objects in a valid state from the beginning, and manager classes group the operations that belong together.

- Classes and objects to model users, admins and complaints.
- Encapsulation, with private fields and controlled access through methods.
- Constructors for creating objects in a consistent state.
- Methods that carry the behaviour of each class.
- Access modifiers to control what is visible outside a class.
- Abstraction, so the menu code can ask for a complaint without knowing how it is stored.
- A modular class-based structure split across model, service and menu packages.

7.2 Java Collections

Collections do the in-memory work. Instead of juggling arrays with a manual counter, the manager classes hold their records in collections and let the framework handle resizing, insertion and iteration.

- ArrayList for maintaining ordered groups of users and complaints.
- HashMap and similar structures where a manager needs to look something up by a key.
- Iteration over these collections for displaying lists and for searching.

7.3 File Handling

Collections only last as long as the program does, so file handling was added to make the data permanent. When the application starts it reads the stored records back into memory, and when something changes it writes the updated records out again. What gets saved is the user details, the complaint details, and each complaint's priority and status.

7.4 Exception Handling

Console input is unpredictable, and a program that crashes the first time someone types a letter where a number was expected is not much use. Exception handling is used to catch these cases and recover from them. For instance, if the priority entered for a complaint is not one of the accepted values, the program does not stop; it assigns a default priority and continues.

7.5 Enums

Priority and status are fixed sets of values, so they are written as enums rather than as strings. This means the compiler enforces the allowed values, invalid entries such as a misspelled priority never make it into the data, and the code reads more clearly at the point where these values are

used.

- Priority: LOW, MEDIUM, HIGH.
- ComplaintStatus: the values representing a complaint's progress, such as PENDING and RESOLVED.

8. Features and Modules

8.1 Main Menu

The main menu is the first thing shown when the program runs. It offers four choices: register as a new user, log in as an existing user, log in as an administrator, or exit.

8.2 User Registration and Login

A new user registers by entering a username, a password and their name. Once registered, they log in with the same credentials, and the system checks them against the stored user records before letting them through.

8.3 User Dashboard

After a successful login the user reaches their dashboard, which is where most of the day-to-day activity happens. From here they can register a new complaint, view the complaints they have filed themselves, view all complaints in the system, search for a complaint by ID, look at the latest complaint, or log out.

8.4 Complaint Registration

Registering a complaint is a short sequence of prompts: a title, a description, a category and a priority. The complaint ID is not asked for, because the system generates it. Once the details are entered, a Complaint object is created, added to the collection and written to the data file.

8.5 Complaint Search

Search works on the complaint ID. The user enters an ID and the system looks through the stored complaints and displays the matching record with all its details, or reports that no such complaint exists.

8.6 Admin Dashboard

The administrator's dashboard is broader than the user's. It allows viewing the list of registered users, viewing all complaints, searching for a particular complaint, updating the status and priority of a complaint, and listing the complaints that are still pending.

8.7 Manager Classes

The manager classes exist so that the menu code stays thin. Every menu option does little more than collect input and call a method on a manager, which keeps the business logic in one place

and makes it possible to test that logic on its own. The managers in the project are ComplaintManager, UserManager, AdminManager, PriorityManager, CategoryManager, DepartmentManager, PendingComplaintManager and FrequentComplainerManager.

9. System Design and Working

9.1 General Working Flow

A typical session runs like this. The application starts and loads whatever users and complaints are already saved on disk, then displays the main menu. A new person registers an account; an existing one logs in directly. After a successful login the user dashboard opens. From there the user registers a complaint by entering its details, and the system immediately generates a complaint ID and shows it, so the user has something to refer to later. The complaint is added to the in-memory collection and written to the data file at the same time. The user can then view or search complaints as needed, and finally log out to return to the main menu. An administrator can log in through the admin option at any point and manage the records from the admin dashboard.

9.2 Complaint Registration Flow

Looking at complaint registration in more detail, the steps are:

- The user selects Register Complaint from the dashboard.
- The system works out the next complaint ID.
- The user is prompted for a title and a description.
- The user enters the category the complaint belongs to.
- The user selects a priority; if the input is not valid, a default is used.
- A Complaint object is created from these values.
- ComplaintManager adds the complaint to its collection.
- The complaint is written to the complaint data file.

9.3 Sample Complaint Record

A stored complaint looks like this when displayed:

```
Complaint ID : 101
Title       : WiFi not working
Category    : Network
Priority     : HIGH
Status      : PENDING
```

10. Project Structure

The repository is laid out so that the purpose of each file is obvious from where it sits. Model classes hold data, service classes hold logic, and the menu classes sit at the top level alongside Main.

```
Smart-Complaint-Management-System/
|
+-- src/
|   +-- complaintmanagement/
|       +-- Main.java
|       +-- UserMenu.java
|       +-- AdminMenu.java
|       +-- model/
|           | +-- Admin.java
|           | +-- Complaint.java
|           | +-- ComplaintStatus.java
|           | +-- Priority.java
|           | +-- User.java
|       +-- service/
|           +-- AdminManager.java
|           +-- CategoryManager.java
|           +-- ComplaintManager.java
|           +-- DepartmentManager.java
|           +-- FrequentComplainerManager.java
|           +-- PendingComplaintManager.java
|           +-- PriorityManager.java
|           +-- UserManager.java
|
+-- data/
+-- .gitignore
+-- README.md
```

11. Data Storage

Records are kept in plain text files inside a data folder, which is created or updated as the application runs. Two files are used:

- data/users.txt, holding the registered user accounts.
- data/complaints.txt, holding every complaint that has been filed.

Complaints are stored one per line, with the fields separated by a pipe character. The order of the fields is:

```
complaintId | title | description | category | username | priority | status
```

So a real line in the file looks like this:

```
101|WiFi not working|Hostel WiFi is not working  
properly.|Network|avani|HIGH|PENDING
```

A pipe was chosen as the separator instead of a comma because complaint descriptions are free text and commas appear in them constantly, which would break the parsing. The format is simple enough to read by hand, which was useful while debugging.

12. Testing

Test classes were written for the important manager operations, and the application was also exercised by hand through the console. The table below lists the cases that were checked and what each one is expected to do.

Test Case	Expected Result
User registration	A new user record is created and saved
User login	Valid credentials allow access to the dashboard
Admin login	Valid credentials open the admin dashboard
Complaint registration	The complaint is added to the collection and the file
Complaint ID generation	Every complaint receives a unique ID
View complaints	Stored complaint details are displayed correctly
Search complaint	The complaint is found using its ID
Invalid priority input	A default priority is assigned instead of crashing
File persistence	Records are still present after restarting the program
Logout	The user is returned to the main menu

13. Advantages

The system has a few clear strengths, most of which come from keeping it simple:

- The console interface is easy to follow, with numbered options at every step.
- Complaint IDs are generated automatically, which removes the duplicate-number problem entirely.
- Records are stored in a consistent format instead of whatever a person happened to write.
- User and admin functions are separate, so a normal user cannot change a complaint's status.
- It is a working demonstration of object-oriented design rather than a set of unrelated programs.
- Collections handle the record management, which keeps the code short and readable.
- Data survives between runs because of file handling.

- The structure makes it easy to add features later without rewriting what is already there.

14. Limitations

It is worth being honest about what the current version does not do:

- It runs only on the console, so there is no graphical interface.
- Data lives in text files rather than a database, which is fine for a few hundred records but would not hold up at a larger scale.
- It runs locally on one machine and is intended for academic use.
- Passwords are stored as plain text; no encryption or hashing has been implemented.
- There are no email or SMS notifications, so users must check the system themselves for updates.
- Concurrent access is not handled, since only one person can run the program at a time.

15. Future Scope

Most of the limitations above point directly at the next steps. The improvements that would make the most difference are:

- A graphical user interface built with Swing or JavaFX.
- A web-based version so complaints can be filed from any device.
- A MySQL database in place of text files, with proper queries instead of line parsing.
- Email or SMS notifications when a complaint's status changes.
- Detailed progress tracking, showing the history of each complaint rather than only its current state.
- An analytics dashboard for the administrator, showing complaint volume by category and average resolution time.
- Filtering and sorting of complaints by category, priority or date.
- Password hashing and stronger authentication.
- Role-based access control, so departments only see the complaints that concern them.
- Export of complaint data to CSV or PDF for reporting.

16. Conclusion

The Smart Complaint Management System does what it set out to do. Complaints can be registered, stored, searched and managed in an organised way, and the information stays available between sessions instead of disappearing when the program closes.

More importantly, from the point of view of the coursework, the project brought several Java concepts together in one place. Object-oriented design decided how the code was structured, collections handled the records, enums kept priority and status valid, exception handling made the program tolerant of bad input, and file handling gave it persistence. Working with all of these at once is quite different from practising them separately, and a fair amount of what I learned came from the parts that did not work the first time.

The current version is console-based and built for academic use, but the structure it uses is not a dead end. Because the logic is kept in manager classes and separated from the display code, replacing the console with a graphical interface, or the text files with a database, would mean changing one layer rather than rewriting the application.

17. References

- Oracle, *Java SE Documentation and API Specification*, Oracle Corporation.
- Oracle, *The Java Tutorials: Collections Trail*.
- Herbert Schildt, *Java: The Complete Reference*, McGraw Hill Education.
- Kathy Sierra and Bert Bates, *Head First Java*, O'Reilly Media.
- Oracle, *The Java Tutorials: Exceptions and Basic I/O*.
- Git and GitHub official documentation.
- Course material and lecture notes provided through the VITyarthi platform.